

**VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY
INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING**

Technical Report

Web Application Development

Topic: U - School Management System

Team Members:

Tran Anh Van	ITITWE22171
Nguyen Tran Minh Thu	ITITWE24025

Instructor: Nguyen Trung Nghia, MS

Contents

1	Executive Summary	5
1.1	Project Overview	5
1.2	Problem Statement	5
1.3	Target User Groups	5
1.4	Key Features	6
1.5	Technology Stack	6
1.6	Deployment and Testing Credentials	6
2	Requirements Analysis	7
2.1	Detailed Problem Statement	7
2.2	User Stories and Acceptance Criteria	7
2.2.1	Domain 1: Identity, Access & Session Management	7
2.2.2	Domain 2: Profile & Directory Management	10
2.2.3	Domain 3: Curriculum & Classroom Allocation	13
2.2.4	Domain 4: Course Delivery & Scheduling	15
2.2.5	Domain 5: Academic Enrollment	19
2.2.6	Domain 6: Finances & Operations	22
2.3	Functional Requirements	26
2.4	Non-Functional Requirements	26
3	System Architecture	27
3.1	High-Level Architecture	27
3.2	Technology Stack Justification	27
3.3	Design Patterns	27
3.4	API Design	28
4	Database Design	30
4.1	Entity-Relationship Diagram (ERD)	30
4.2	Table Schemas	30
4.3	Database Normalization (3NF)	31
4.4	Indexing Strategy	32
5	Implementation Details	33
5.1	Key Algorithms and Business Logic	33
5.1.1	Schedule Conflict Detection	33
5.2	Security Implementation	33
5.3	Error Handling Strategy	34
5.4	AI Assistance Report	34
5.4.1	AI Assistance Summary	34

5.4.2	Major Components Where AI Was Used	34
6	Testing and Quality Assurance	35
6.1	Testing Strategy	35
6.2	Test Scenarios	35
6.3	Known Limitations	35
7	Deployment	36
7.1	Deployment Process	36
7.1.1	Frontend Hosting	36
7.1.2	Backend Hosting	36
7.2	Environment Variables	36
8	User Guide	37
8.1	Getting Started and Authentication	37
8.2	Course Registration and Searching	38
8.3	Weekly Class Schedule Timetable	38
8.4	Advising Approval Walkthrough	39
9	Team Collaboration	40
9.1	Team Organization	40
9.2	Individual Contributions	40
10	Conclusion and Reflection	41
10.1	Project Summary	41
10.2	Lessons Learned	41
A	Meeting Logs	42
B	API Documentation (OpenAPI 3.1.0 Specification)	43

List of Figures

3.1	Use Case Diagram	29
4.1	Entity-Relationship Diagram (ERD)	30
8.1	Edusoft Portal Authentication Screen	37
8.2	Course Registration Panel with Course Offerings and Selected Drafts . .	38
8.3	Personal Weekly Class Schedule Timetable Grid	39

List of Tables

1.1	Pre-seeded Evaluation Accounts	6
6.1	Selected Verification Scenarios	35

Chapter 1

Executive Summary

1.1 Project Overview

Edusoft is a comprehensive, production-grade School Management System (SMS) built to streamline administration and academic registry operations at the university level. Academic institutions frequently grapple with disparate administrative systems, manual registrar validations, and latency in academic advisor sign-offs. Edusoft addresses these operational challenges by consolidating key workflows under a unified, role-based single-page application (SPA).

1.2 Problem Statement

In traditional academic environments, student registration, financial ledger adjustments, and advising approvals operate as disconnected systems. This isolation introduces manual inefficiencies, such as advisors reviewing course requests without real-time schedule conflict checks or credit caps. Furthermore, delayed tuition invoice generation leads to non-compliant student enrollments. Edusoft resolves these operational bottlenecks through a reactive, transaction-safe database model that enforces constraints at the engine level.

1.3 Target User Groups

The application supports three primary user groups, each granted unique access scopes:

- **Administrators:** Supervise account provisioning, manage the core curriculum (Subjects, Rooms, and Course Sections), and oversee system configuration parameters.
- **Academic Advisors (Staff):** Access student advising profiles, evaluate pending registrations grouped by student, view teaching timetables, and audit payroll histories.
- **Students:** Request course registrations, view personalized weekly timetables, and audit financial statements or overdue tuition fees.

1.4 Key Features

The core capabilities of the system include:

- **Identity and Directory Management:** Complete CRUD directory interface for Users, Students, and Staff with bulk CSV import operations wrapped in all-or-nothing transactions.
- **Curriculum Configuration:** System-wide setup for subjects, rooms, and course sections with registration window restrictions.
- **Advising Workflow:** Custom grouped approval screens for academic advisors to approve or decline student registration requests with inline comment feedback.
- **Timetable Verification:** Interactive calendars that filter and hide schedule details outside the active semester date range.
- **Automated Financial Ledger:** Dynamic tuition fee calculation linked directly to approved course enrollment credits, automatically adjusting balances during registration drops.

1.5 Technology Stack

The application leverages a modern, unified JavaScript/TypeScript stack:

- **Frontend:** React 18, Vite, Tailwind CSS, TypeScript, and React Query (TanStack Query) for declarative data caching.
- **Backend:** Node.js, Express, TypeScript, and Prisma ORM.
- **Database:** MySQL relational database engine.

1.6 Deployment and Testing Credentials

The system is deployed on public cloud services:

- **Frontend URL:** <https://school-management-system-hcmiu.vercel.app/>
- **Backend URL:** <https://school-management-system-backend.up.railway.app/>

For evaluation, the following pre-seeded credentials are provided for local and staging access:

Role	Email Address	Default Password
Admin	admin2@hcmiu.edu.vn	Password123!
Staff (Advisor)	nhphu@hcmiu.edu.vn	Password123!
Student	ITITWE24025@student.hcmiu.edu.vn	Password123!

Table 1.1: Pre-seeded Evaluation Accounts

Chapter 2

Requirements Analysis

2.1 Detailed Problem Statement

Higher education administration demands high operational accuracy. Key challenges solved by the Edusoft system include:

- **Concurrent Capacity Management:** Over-enrollment in course sections due to race conditions during high-volume registration periods.
- **Compliance Checks:** Enforcing financial blocks (overdue fees) and credit ceilings dynamically before allowing registration requests.
- **Advisor Overhead:** Standard tools display student registration requests on a course-by-course basis. Advisors require a grouped student-centric card layout to review a student's full term load.

2.2 User Stories and Acceptance Criteria

The application implementation fulfills the comprehensive set of user stories and acceptance criteria. Below, these are detailed domain-by-domain:

This document outlines the system requirements and specifications for the School Management System (SMS). The user stories are organized into 6 core business domains, designed for single-responsibility and complete test coverage with Gherkin-style Acceptance Criteria.

2.2.1 Domain 1: Identity, Access & Session Management

SMS-1: User Account Directory (Admin)

As an Admin,

I want to view and manage base user accounts,

so that I can supervise access, search for users, and prevent duplicate registration.

Scenario 1: Paginated User Listing

- **Given** an Admin is authenticated,

- **When** they request the user list with parameters (e.g., 'page=1', 'limit=20'),
- **Then** the backend must return the current page of users along with pagination metadata ('totalCount', 'totalPages', 'currentPage').

Scenario 2: Filter and Search Directory

- **Given** an Admin is viewing the user list,
- **When** they filter by role (Admin, Staff, Student), status (Active where 'is_active = true'; Inactive where 'is_active = false' and password is set; Pending where 'is_active = false' and password is not set), or search by email,
- **Then** the system must return a filtered list matching all criteria simultaneously.

Scenario 3: Create Base User Account

- **Given** an Admin is creating a user account,
- **When** they submit an email and select a role,
- **Then** the system must verify the email is unique (case-insensitive check),
- **And** create the user with 'is_active = false' (representing the 'pending_activation' state),
- **And** return a '201 Created' status with the generated onboarding link/token,
- **And** reject the request with a '409 Conflict' if the email is already in use.

Scenario 4: Role Mutation Safeguards

- **Given** a User account is linked to an active 'Staff' or 'Student' profile,
- **When** an Admin attempts to change that User's role to a different role,
- **Then** the backend must reject the update with a '409 Conflict' status, requiring the Admin to delete or unlink the associated profile first to prevent orphaned profile records.

Scenario 5: User Account Deactivation/Suspension by Admin

- **Given** an Admin is viewing a User's account in the directory,
- **When** they request to deactivate the User account,
- **Then** the backend must set 'is_active = false' on the User record,
- **And** immediately invalidate and clear all active JWT session tokens for that user, preventing further API access.

Scenario 6: Admin Self-Harm Guardrails

- **Given** an Admin is logged in,
- **When** they attempt to deactivate their own account, change their own role, or delete their own user account,
- **Then** the backend must reject the request with a '400 Bad Request' status and return a message: "Self-deactivation, self-demotion, or self-deletion is not permitted."

Scenario 7: User Account Hard Deletion Cascade

- **Given** a Student or Staff profile is hard-deleted from the database,
- **When** the deletion transaction runs,
- **Then** the backend must hard-delete the corresponding base User record in the same database transaction.

Scenario 8: Onboarding Token Resending

- **Given** a User account is in the pending activation state,
- **When** an Admin requests to resend the onboarding activation link,
- **Then** the backend must invalidate any existing onboarding token for that user,
- **And** generate a new secure activation token valid for 72 hours,
- **And** return a '200 OK' status with the new token or trigger an automated email dispatch containing the new link.

Scenario 9: Administrative Action Audit Logging

- **Given** an Admin performs a mutating action in the User Directory (creating a user, deactivating/reactivating an account, changing a role, manually unlocking a user, or resending onboarding tokens),
- **When** the database update is committed,
- **Then** the backend must write an entry to the 'Audit_Log' containing the Admin's user ID, target user ID, action name, old and new values, client IP address, and timestamp.
- –

2.2.2 Domain 2: Profile & Directory Management

SMS-2: Staff Directory & Profile Management

As an Admin,

I want to manage staff profiles and departmental records,
so that the school's employee directory is accurate and audit-compliant.

Scenario 1: Link Staff Profile

- **Given** a base User account with the 'Staff' role has been created,
- **When** the Admin submits a unique 'employee_code' (trimmed and non-empty), 'first_name', 'last_name', 'department', and 'hire_date' linked to that User,
- **Then** the backend must verify the base User account is not already linked to another staff profile (returning a '409 Conflict' if it is),
- **And** create the Staff profile, returning a '201 Created' status.

Scenario 2: Unique Employee Code Enforcement

- **Given** an Admin is creating or updating a Staff profile,
- **When** they submit an 'employee_code' that is already assigned to another staff member,
- **Then** the backend must reject the payload and return a '409 Conflict' status.

Scenario 3: Paginated Staff Directory and Filtering

- **Given** an Admin is viewing the staff list,
- **When** they query the directory with search filters (name, employee code, department),
- **Then** the backend must return a paginated result containing only matching records.

Scenario 4: Bulk Staff Import via CSV

- **Given** an Admin uploads a CSV containing staff details,
- **When** the backend parses the file,
- **Then** it must execute the entire import inside a single parent database transaction (ensuring an all-or-nothing rollback for all rows),
- **And** check for duplicate emails or employee codes within the CSV file itself before database processing (returning a '400 Bad Request' list of duplicate errors if any are found),

- **And** check for employee code and email duplicates against existing database records,
- **And** create a base User account for each staff member with status ‘pending_activation’, generating a temporary password and a secure onboarding activation token,
- **And** create the corresponding Staff profile,
- **And** roll back all changes completely if any row fails validation (e.g. invalid email format, duplicate code), returning a ‘400 Bad Request’ status with a list of specific row errors.

Scenario 5: Soft Delete/Deactivation Consequence

- **Given** a Staff member is teach-assigned to a course section,
- **When** an Admin deactivates their profile,
- **Then** the backend must set their user ‘is_active’ status to false,
- **And** invalidate their active sessions,
- **And** keep historical assignments intact, but block them from being assigned to any new course sections.

Scenario 6: Staff Deletion and Audit Integrity

- **Given** a Staff member exists,
- **When** the Admin attempts to delete the Staff profile,
- **Then** the backend must check if the Staff member is currently assigned to teach any active course section in the current semester, rejecting the delete with a ‘409 Conflict’ status if found,
- **Or** if they have historical teaching records OR salary records, perform a soft delete by deactivating their user account to preserve database audit history,
- **Or** if they have no historical teaching or salary records, perform a hard delete of the staff profile and their base user account from the database.
- –

SMS-3: Student Directory & Profile Management

As an Admin or Staff member,
I want to view, search, and manage student profiles,
so that learner directory records are accurate and searchable.

Scenario 1: Link Student Profile

- **Given** a base User account with the ‘Student’ role has been created,

- **When** the Admin submits a unique ‘student_code’, ‘first_name’, ‘last_name’, ‘class_level’, ‘enrollment_date’, and a valid ‘advisor_id’ (referencing an active Staff member) linked to that User,
- **Then** the backend must verify the ‘advisor_id’ exists and is active (returning a ‘400 Bad Request’ if invalid or deactivated),
- **And** verify the base User account is not already linked to another student profile (returning a ‘409 Conflict’ if it is),
- **And** create the Student profile, returning a ‘201 Created’ status.

Scenario 2: Advanced Multi-Criteria Search

- **Given** an Admin or Staff member is on the Student directory page,
- **When** they submit a search query filtering by name/code, class level (year), and tuition fee status (Paid, Unpaid, Overdue) simultaneously,
- **Then** the backend must query the database matching all active filters and return the paginated list.

Scenario 3: Bulk Student Import via CSV

- **Given** an Admin uploads a student CSV import file,
- **When** the system runs the import,
- **Then** it must execute within a single parent database transaction (ensuring an all-or-nothing rollback for all rows),
- **And** check for duplicate emails or student codes within the CSV file itself before database processing (returning a ‘400 Bad Request’ list of duplicate errors if any are found),
- **And** check for student code and email duplicates against existing database records,
- **And** create a base User account for each student with status ‘pending_activation’, generating a temporary password and a secure onboarding activation token,
- **And** create the corresponding Student profile,
- **And** roll back all changes completely if any row is invalid, returning a ‘400 Bad Request’ status with a detailed row-by-row validation report.

Scenario 4: Soft Delete & Financial Ledger Constraint

- **Given** a Student has active or historical ‘Fee’ records OR enrollment records in the system,
- **When** an Admin attempts to delete the student profile,

- **Then** the backend must reject hard deletion and return a ‘409 Conflict’ status,
- **And** require a soft delete instead by setting the user’s ‘is_active’ status to false, which preserves financial audit and academic histories.

Scenario 5: Academic Advisor Reassignment

- **Given** a Student has active ‘pending’ enrollment requests,
- **When** an Admin updates the Student’s profile to assign a new ‘advisor_id’,
- **Then** the backend must verify that the new advisor exists in the ‘Staff’ table and is active (‘Users.is_active === true’), returning a ‘400 Bad Request’ status if inactive or missing,
- **And** update the advisor reference in the Student record,
- **And** instantly transfer the approval/decline authorization for all active ‘pending’ enrollments of that student to the new advisor.
- –

2.2.3 Domain 3: Curriculum & Classroom Allocation

SMS-4: Subject Catalog Management

As an Admin,

I want to configure curriculum subjects,

so that courses can be officially offered and credits counted.

Scenario 1: Create Catalog Subject

- **Given** an Admin is configured on the Academic settings panel,
- **When** they submit a unique ‘subject_code’, ‘name’, and ‘credits’ (an integer),
- **Then** the backend must query the global ‘MAX_SUBJECT_CREDITS’ config (defaulting to 10),
- **And** save the subject and return a ‘201 Created’ status if credits are between 1 and ‘MAX_SUBJECT_CREDITS’ inclusive,
- **And** reject credit values outside this range or non-unique codes with a ‘400 Bad Request’ status.

Scenario 2: Modification Restrictions on Active Terms

- **Given** a Subject is referenced by an active ‘Course_Section’ in the current active semester,
- **When** an Admin attempts to edit its code or credits,

- **Then** the backend must block the edit and return a ‘409 Conflict’ status to prevent altering active enrollment calculations.

Scenario 3: Subject Deletion and Soft Delete

- **Given** a Subject exists,
- **When** the Admin requests to delete the Subject,
- **Then** the backend must check if the Subject has historical or active course sections,
- **And** if it does, perform a soft delete by setting ‘is_active’ to false, blocking scheduling of new sections while preserving transcripts,
- **Or** if it has zero associated course sections, perform a hard delete from the database.

Scenario 4: Reactivate Soft-Deleted Subject

- **Given** a Subject is soft-deleted (‘is_active’ is false),
- **When** the Admin requests to reactivate the Subject,
- **Then** the backend must set ‘is_active’ to true,
- **And** allow new course sections to be scheduled for it again.
- –

SMS-5: Classroom & Infrastructure Allocation

As an Admin,

I want to manage physical classrooms and labs,

so that class scheduling matches room type and capacity.

Scenario 1: Register Room

- **Given** an Admin is on the Infrastructure page,
- **When** they submit a ‘room_number’ (trimmed and non-empty), ‘capacity’ (integer 1-250), and ‘is_lab’ flag,
- **Then** the system must validate the fields, returning a ‘400 Bad Request’ if bounds or formats are invalid,
- **And** save the room, enforcing unique room numbers (case-insensitive check) and returning ‘201 Created’.

Scenario 2: Room Capacity and Lab Suitability Adjustments Check

- **Given** an Admin is updating a Room’s capacity or type,

- **When** they attempt to lower the capacity below the enrollment count OR below the ‘max_capacity’ limit of any active course section scheduled in that room for the current semester,
- **Then** the backend must reject the update and return a ‘409 Conflict’ status to prevent classroom over-enrollment,
- **Or When** they attempt to update ‘is_lab’ to false for a room that currently hosts active schedule slots of any lab-requiring subject in the current semester,
- **Then** the backend must reject the update and return a ‘409 Conflict’ status to prevent course lab suitability violations.

Scenario 3: Room Maintenance Mode & Block

- **Given** an Admin is placing a room into maintenance mode (‘is_active’ set to false),
- **When** there are active schedules using this room in the current semester,
- **Then** the backend must block the deactivation and return a ‘409 Conflict’ status, displaying the list of affected course sections,
- **And** require the Admin to reschedule or delete those active schedule slots before the room can be successfully set to inactive.
- –

2.2.4 Domain 4: Course Delivery & Scheduling

SMS-6: Manage Course Sections Offering

As an Admin,

I want to establish course sections for subjects in active semesters,
so that students can enroll in specific classes.

Scenario 1: Create Course Section

- **Given** an Admin is creating a class offering,
- **When** they select a Subject, a Staff member (Teacher), ‘semester’ (‘1’, ‘2’, or ‘3’), ‘academic_year’ (between 1900 and 2100), ‘section_number’ (trimmed and non-empty), and ‘max_capacity’ (greater than 0),
- **Then** the backend must verify the Subject and Staff member exist and are active (‘is_active’ is true), returning ‘400 Bad Request’ if either is inactive,
- **And** verify that the section’s ‘max_capacity’ does not exceed the global ‘MAX_ROOM_CAPACITY’ setting (returning a ‘400 Bad Request’ if it does),
- **And** verify that the selected ‘semester’ and ‘academic_year’ are greater than or equal to the system’s current active semester and academic year settings, blocking creation of sections in past terms with a ‘400 Bad Request’ status,

- **And** block duplicates of the same subject-semester-year-section code with a ‘409 Conflict’ status,
- **And** create the section.

Scenario 2: Capacity Reductions and Limits Check

- **Given** an Admin is updating a course section,
- **When** they reduce the ‘max_capacity’ below the number of students already enrolled (approved or pending),
- **Then** the backend must reject the request with a ‘409 Conflict’ status,
- **Or When** they update the ‘max_capacity’ to a value that exceeds the global ‘MAX_ROOM_CAPACITY’ setting, OR attempt to set the ‘semester’ or ‘academic_year’ to a past term relative to current active configurations,
- **Then** the backend must reject the request with a ‘400 Bad Request’ status.

Scenario 3: Deletion Restrictions

- **Given** a Course Section exists,
- **When** the Admin attempts to delete the section,
- **Then** the backend must verify if there are one or more registered students (pending or approved) and reject the deletion with a ‘409 Conflict’ status if found,
- **Or** if no students are registered, delete the section along with all its scheduled time slots in a single transaction (Cascade Delete schedules).

Scenario 4: Section Teacher Reassignment & Exclusions

- **Given** a Course Section has scheduled time slots,
- **When** the Admin updates the Course Section to assign a new Staff member (Teacher),
- **Then** the backend must verify the new Teacher exists, is active, and has no overlapping teaching schedules in the database for the active semester (excluding schedules of this specific Course Section if the teacher remains unchanged),
- **And** reject the update with a ‘409 Conflict’ status if a schedule overlap conflict is detected.
- –

SMS-7: Class Time & Room Scheduling

As an Admin,

I want to schedule time slots and room allocations for course sections, so that room double-bookings and staff schedule overlaps are blocked.

Scenario 1: Schedule Time Slot

- **Given** a course section exists,
- **When** the Admin assigns a 'day_of_week' (MON, TUE, etc.), 'start_time' (HH:MM:SS), 'end_time' (HH:MM:SS), and 'room_id',
- **Then** the backend must verify 'start_time' is before 'end_time',
- **And** commit the schedule if no conflicts exist,
- **And** when updating an existing schedule, the conflict detection queries must exclude the current schedule ID to prevent self-conflict false positives.

Scenario 2: Room Double-Booking Clash Block

- **Given** Room A is scheduled on Monday from 09:00 to 11:00 in Semester 1 (2026),
- **When** the Admin attempts to assign Room A on Monday to another section in Semester 1 (2026) with overlapping times (e.g., 10:00 to 12:00),
- **Then** the backend must reject the schedule with a '409 Conflict' status,
- **But When** the Admin attempts to assign Room A on Monday to an overlapping time in Semester 2 (2026),
- **Then** the backend must allow the schedule to be committed.

Scenario 3: Staff Double-Booking Clash Block

- **Given** Teacher A is teaching a section scheduled on Tuesday from 13:00 to 15:00 in Semester 1 (2026),
- **When** the Admin attempts to schedule another section taught by Teacher A on Tuesday at an overlapping time (e.g., 14:00 to 16:00) in Semester 1 (2026),
- **Then** the backend must reject the schedule with a '409 Conflict' status,
- **But When** the Admin attempts to schedule Teacher A on Tuesday at an overlapping time in Semester 2 (2026),
- **Then** the backend must allow the schedule to be committed.

Scenario 4: Classroom Lab Suitability Enforcement

- **Given** a Subject requires a laboratory setting ('requires_lab' is true),
- **When** the Admin attempts to schedule a section of this subject in a Room where 'is_lab' is false,
- **Then** the backend must reject the request and return a '400 Bad Request' status.

Scenario 5: Classroom Capacity Fit Warning

- **Given** Room A has a capacity of 30, and Course Section B has a 'max_capacity' of 50,
- **When** the Admin attempts to allocate Room A to Course Section B,
- **Then** the backend must reject the request and return a '400 Bad Request' status to ensure classroom size fits the section's maximum possible enrollment.

Scenario 6: Inactive Infrastructure, Subject, and Staff Safeguards

- **Given** a Room has been marked as inactive/maintenance mode ('is_active' set to false), a Subject is inactive ('is_active' set to false), or the assigned Teacher of a Course Section is deactivated ('is_active' of their linked User profile is false),
- **When** the Admin attempts to schedule a new time slot in that Room, or for a section of that Subject or Teacher,
- **Then** the backend must reject the schedule creation or update and return a '400 Bad Request' status.
- –

SMS-8: Staff Class Schedule & Portal

As a Staff member,

I want to view my teaching schedule,

so that I know when and where to teach.

Scenario 1: Assigned Course Offerings

- **Given** a logged-in Staff member (Teacher),
- **When** they request their course list,
- **Then** the system must return only the course sections where they are assigned as the teacher for the current active semester.

Scenario 2: Weekly Teaching Timetable Grid

- **Given** a logged-in Staff member,

- **When** they view their schedule dashboard,
- **Then** the frontend must display a weekly grid layout displaying scheduled day, times, room numbers, and subject codes for all their assigned sections.

Scenario 3: Timetable Data Access Control

- **Given** an authenticated user is requesting a Staff member's weekly timetable data via the API,
- **When** the request is received by the backend,
- **Then** the backend must verify the requester is either an Admin or the target Staff member,
- **And** reject the request and return a '403 Forbidden' status if they are unauthorized.
- –

2.2.5 Domain 5: Academic Enrollment

SMS-9: Course Registration & Enrollment Compliance

As a Student,

I **want to** register for courses that match my curriculum and calendar,

As an Academic Advisor (Staff),

I **want to** approve or decline pending student enrollments for my advisees,
so **that** enrollment compliance and advisement guidelines are enforced.

Scenario 1: Enrollment Request Submission

- **Given** an active Student is logged in during the active registration window,
- **When** they submit a request to enroll in a Course Section,
- **Then** the system must execute the capacity checks, credit calculations, and registration creation inside a database transaction lock (e.g. select-for-update or serializable transaction) to prevent concurrent request race conditions,
- **And** verify the section is not at or above 'max_capacity',
- **And** verify the student is not already enrolled,
- **And** verify the additional credits do not push their semester credits over the 'MAX_SEMESTER_CREDITS' config (e.g., 18 credits),
- **And** create a 'Student_Enrollment' record with a status of 'pending'.

Scenario 2: Student Timetable Conflict Block

- **Given** a student is enrolling in a section,

- **When** the section schedule overlaps with any of their currently approved or pending registrations,
- **Then** the backend must block enrollment and return a '409 Conflict' status.

Scenario 3: Duplicate Subject Block

- **Given** a student is enrolled in Section A of Subject "CS101",
- **When** they attempt to enroll in Section B of Subject "CS101" in the same semester,
- **Then** the backend must reject the enrollment and return a '409 Conflict' status.

Scenario 4: Tuition Compliance Block (Overdue Fees)

- **Given** a Student has one or more fee statements marked 'overdue' (unpaid past due date),
- **When** they attempt to submit an enrollment request,
- **Then** the backend must block the registration and return a '402 Payment Required' status.
- **And** immediately lift this block once the outstanding fee balance is marked as 'paid'.

Scenario 5: Enrollment Decision Workflow Restricted to Academic Advisor

- **Given** a Student has an assigned Academic Advisor (a Staff member),
- **When** a user attempts to approve or decline a pending student enrollment,
- **Then** the backend must verify the user is the Student's designated Academic Advisor,
- **And** reject the request with a '403 Forbidden' status if the requester is any other user (including the course teacher, administrators, or other staff members),
- **Or** update the enrollment status to 'approving' (Approved) or 'declining' (Declined) in the database,
- **And** adjust the course section's active enrollment counter accordingly if approved.

Scenario 6: Listing Advisee Pending Enrollments

- **Given** an Academic Advisor (Staff) is logged in,
- **When** they request the list of pending student enrollments,
- **Then** the backend must query the database and return only pending enrollments for students whose assigned 'advisor_id' matches the logged-in staff member's ID,

- **And** exclude any student enrollments for students assigned to other advisors.

Scenario 7: Drop Course Section during Registration Window

- **Given** a Student is logged in and has an active enrollment (pending or approved) in a course section,
- **When** they request to drop the course section,
- **Then** the backend must verify the current system date falls within the active registration window,
- **And** reject the request with a '400 Bad Request' status if submitted outside the active registration window dates,
- **Or** delete/remove the student's enrollment record from the database,
- **And** decrement the course section's active enrollment count,
- **And** trigger an automatic tuition recalculation to refund/credit the student's semester invoice.
- –

SMS-10: Student Personal Portal & Timetable

As a Student,

I want to view my personal weekly timetable,
so that I know my weekly schedule.

Scenario 1: Personal weekly timetable

- **Given** a logged-in Student,
- **When** they request their weekly timetable page data,
- **Then** the backend must return schedules only for sections where the student's enrollment status is 'approved' ('approving'),
- **And** if no approved enrollments exist, return an empty array,
- **And** the frontend must gracefully display a message: "No approved classes scheduled for this week. Please check your pending registrations."

Scenario 2: Timetable Data Access Control

- **Given** an authenticated user is requesting a student's weekly timetable data via the API,
- **When** the request is received by the backend,

- **Then** the backend must verify the requester is either an Admin, the Student owner, or the Student's designated Academic Advisor (Staff),
- **And** reject the request and return a '403 Forbidden' status if they are unauthorized.
- –

2.2.6 Domain 6: Finances & Operations

SMS-11: Student Tuition Billing & Payments

As a Student, I want to view my fee statements and submit payments,
As an Admin, I want to generate bills and track tuition collections,
so that the school's tuition collections are accurate and managed.

Scenario 1: Term Billing Generation

- **Given** an Admin initiates the billing run for the current semester,
- **When** the billing script executes,
- **Then** it must calculate fees for each student based on their approved enrollments (approved credits * cost-per-credit),
- **And** verify the student does not already have an invoice for the current active semester and academic year (skipping generation or returning a warning if it exists to prevent double-billing),
- **Or** create a 'Fee' invoice for each student with status 'unpaid' and a defined 'due_date'.

Scenario 2: Personal Fee Ledger

- **Given** a logged-in Student,
- **When** they visit their financial portal,
- **Then** they must see all invoices showing amount due, amount paid, due date, and payment status (paid, unpaid, overdue).

Scenario 3: Recording Payment Transaction

- **Given** an Admin is recording a payment or a gateway webhook is received,
- **When** the payment details are submitted against an invoice,
- **Then** the backend must verify the gateway's cryptographic signature (if it is a webhook request) to prevent payment spoofing, rejecting invalid payloads with a '401 Unauthorized' status,

- **And** verify the payment amount does not exceed the outstanding balance ('amount_due - amount_paid'),
- **And** reject the request with a '400 Bad Request' status if an overpayment is submitted,
- **Or** record the payment by incrementing the invoice 'amount_paid',
- **And** set the invoice 'updated_by' field to the operator's/gateway's user ID to serve as the transaction audit record,
- **And** set the status to 'paid' if the outstanding balance is zero.

Scenario 4: Automatic Overdue Status Transitions

- **Given** a student fee statement is unpaid,
- **When** the daily system scheduler runs and the current date exceeds the invoice 'due_date',
- **Then** the system must update the invoice payment status to 'overdue'.

Scenario 5: Partial Payments

- **Given** a student pays an amount less than the total outstanding balance,
- **When** the payment transaction is submitted,
- **Then** the system must record the transaction, increment 'amount_paid', keep the status as 'unpaid' (or 'overdue' if past due date), and display the remaining balance.

Scenario 6: Billing Add/Drop Adjustments

- **Given** a student drops a course section during the active registration window,
- **When** the drop is finalized,
- **Then** the system must automatically recalculate their tuition invoice for the semester, reducing the 'amount_due' by the credit value of the dropped course,
- **And** if the student has already paid an amount that exceeds the new recalculated 'amount_due', keep the 'amount_paid' intact, resulting in a negative outstanding balance that indicates an account credit to be manually refunded by admins.

Scenario 7: Billing Operations Authorization

- **Given** a user is logged in with the role of Staff or Student,
- **When** they attempt to trigger a semester billing run or record a payment transaction on any invoice,
- **Then** the backend must reject the request and return a '403 Forbidden' status.
- –

SMS-12: Staff Payroll & Payslip Management

As a Staff member, I want to view my salary statement history,
As an Admin, I want to run monthly payroll and issue payslips,
so that salary processing is correct and transparent.

Scenario 1: Generate Monthly Payslip

- **Given** an Admin is on the Payroll screen,
- **When** they submit 'base_salary', 'allowances', 'deductions', 'payment_month', and 'payment_year' for a Staff member,
- **Then** the backend must verify that the input values for 'base_salary', 'allowances', and 'deductions' are greater than or equal to zero (rejecting the request with a '400 Bad Request' if any input is negative),
- **And** compute 'net_salary = base_salary + allowances - deductions',
- **And** verify that the computed 'net_salary' is greater than zero,
- **And** reject the request with a '400 Bad Request' status if the calculated net salary is zero or negative (due to excessive deductions),
- **Or** save the record in 'Draft' status,
- **And** prevent duplicate entries for the same staff member and month with a '409 Conflict'.

Scenario 2: Payslip Disbursal Workflow

- **Given** salary records are in 'Draft' status,
- **When** the Admin approves and disburses payroll,
- **Then** the status transitions to 'disbursed',
- **And** the payslips become visible to the respective Staff members in their portals.

Scenario 3: Personal Salary Statements History

- **Given** a logged-in Staff member,
- **When** they view their earnings portal,
- **Then** they must see a chronological list of their disbursed salary records,
- **And** the system must block access if they try to view another employee's salary history ('403 Forbidden').

Scenario 4: Payroll Administration Authorization

- **Given** a user is logged in with the role of Staff or Student,
- **When** they attempt to generate a monthly payslip or disburse any payroll,
- **Then** the backend must reject the request and return a '403 Forbidden' status.
- –

SMS-13: System Policy & Settings Configuration

As an Admin,

I want to dynamically configure global limits and active school term parameters,
so that school policies are enforced at runtime without restarting the server.

Scenario 1: Dynamic Policies and Verification

- **Given** an Admin is on the Settings panel,
- **When** they update limits (e.g. 'MAX_FAILED_LOGIN_ATTEMPTS' between 1-20, 'MAX_SEMESTER_CREDITS' between 1-30), change the global active term ('CURRENT_SEMESTER' '1', '2', or '3' and 'CURRENT_ACADEMIC_YEAR'), or update the course registration window ('REGISTRATION_START_DATE' and 'REGISTRATION_END_DATE'),
- **Then** the system must validate the input values conform to constraints,
- **And** verify that 'REGISTRATION_START_DATE' is less than or equal to 'REGISTRATION_END_DATE' in YYYY-MM-DD format (rejecting the save with a '400 Bad Request' if the start date falls after the end date),
- **And** save the changes to the database,
- **And** log the admin's user ID in the audit trail ('updated_by' metadata),
- **And** apply these policy changes instantly to all runtime validation filters.

Scenario 2: Non-Admin Access Rejection

- **Given** a user is logged in with the role of Staff or Student,
- **When** they attempt to read or modify the global configurations,
- **Then** the backend must reject the request and return a '403 Forbidden' status.

Scenario 3: Active Term Configuration & Historical Safeguards

- **Given** the system contains historical grade records, fee invoices, and student schedules,
- **When** the Admin updates the 'CURRENT_SEMESTER' or 'CURRENT_ACADEMIC_YEAR' configs,

- **Then** the system must successfully apply this setting to all future course offerings and registrations,
- **And** must not alter, recalculate, or delete any historical or active student transcripts, tuition fee invoices, or timetables associated with previous semesters, preserving audit history integrity.

2.3 Functional Requirements

- **FR-1:** The system must calculate tuition fees based on the credit weight of approved course enrollments.
- **FR-2:** The system must automatically subtract tuition balance and update the fee status when an approved course is dropped.
- **FR-3:** The system must check for timetable overlaps before adding a course to the student's registration draft.

2.4 Non-Functional Requirements

- **NFR-1 (Security):** All passwords must be stored using BCrypt encryption with a minimum salt round factor of 10.
- **NFR-2 (Availability):** The API backend must implement a rate limiter to block denial-of-service (DoS) attempts.
- **NFR-3 (Performance):** Database writes for student registration must execute inside locked transactions using 'SELECT FOR UPDATE' to avoid double-booking.

Chapter 3

System Architecture

3.1 High-Level Architecture

Edusoft adopts a modern, decoupled client-server architecture. The browser loads the React single-page application, which communicates with the backend Node.js Express server via JSON REST APIs.

3.2 Technology Stack Justification

- **React vs. Spring Boot (Frontend/Backend Choice):** We chose a React SPA over a traditional monolithic Spring Boot MVC framework. React allows for state-driven rendering, which is essential for responsive interactive elements like the weekly calendar grid and inline advisor comment submissions.
- **Express + Node.js vs. Java/Spring Boot:** Express provides a lightweight, highly customizable server framework. When paired with TypeScript, it ensures static type safety across schemas and requests without the overhead and boot latency associated with JVM-based environments.
- **MySQL + Prisma ORM vs. MongoDB:** The relational model is crucial for an academic system. Student-to-advisor relationships, course schedules, and enrollment statuses are highly structured. Relational foreign key constraints and atomic transactions are required to maintain database sanity, which is harder to achieve in a document-based store like MongoDB.

3.3 Design Patterns

- **Repository / Database Abstraction:** Implemented through Prisma ORM, which isolates raw SQL query generation from service controllers.
- **MVC Pattern:** The backend structure separates business modules into routes (V), controllers (C), and services/models (M).
- **Singleton Pattern:** The database client connection pool configuration inside `[db.ts](file:///Users/dylantran/Documents/dev/school-management-system/backend/src/config/db.ts)` behaves as a singleton.

3.4 API Design

The API is designed following RESTful patterns. The prefix ‘/api/v1’ separates versions. Key endpoints include:

- **POST /api/v1/auth/login** - User authentication, returning cookie-based JWT sessions.
- **GET /api/v1/course-sections** - List active course offerings with pagination and query parameters.
- **PATCH /api/v1/enrollments/:id/status** - Update student enrollment state (Approving/Declining).

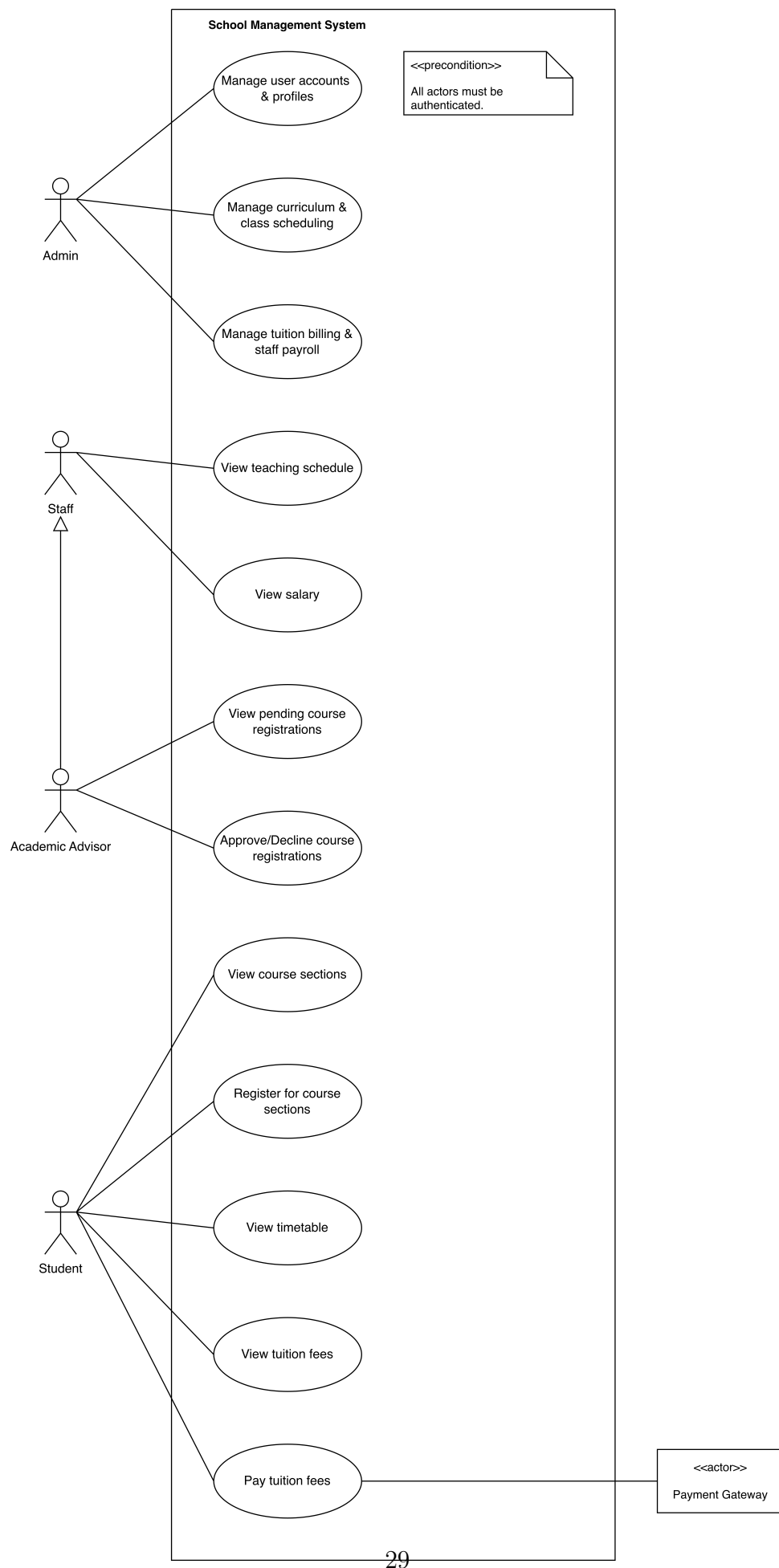


Figure 3.1: Use Case Diagram

Chapter 4

Database Design

4.1 Entity-Relationship Diagram (ERD)

The relational schema comprises 11 tables. Relationships are designed using strong foreign keys, cascading deletions, and uniqueness blocks.

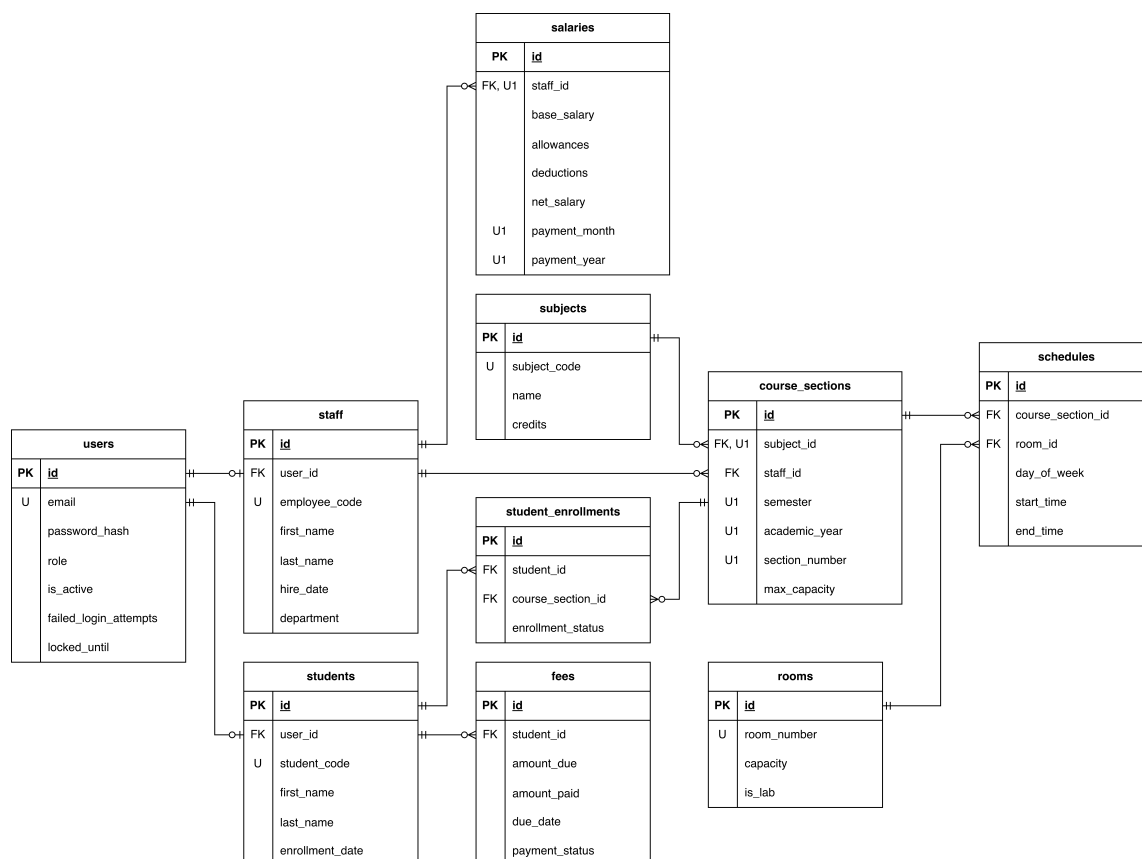


Figure 4.1: Entity-Relationship Diagram (ERD)

4.2 Table Schemas

Key schema definitions are listed below:

- **Users:** Holds credential hashes, roles ('Admin', 'Staff', 'Student'), and active status flags.
- **Student:** Links a user ID to student details, class level, and their designated academic advisor ID.
- **Staff:** Links a user ID to staff credentials and departments.
- **Student_Enrollment:** Junction table connecting Students to Course Sections. Holds the status ('pending', 'approving', 'declining') and optional advisor comments.

```
1 CREATE TABLE Users (  
2   id VARCHAR(36) PRIMARY KEY,  
3   email VARCHAR(100) UNIQUE NOT NULL,  
4   password_hash VARCHAR(255) NOT NULL,  
5   role ENUM('Admin', 'Staff', 'Student') NOT NULL,  
6   is_active BOOLEAN DEFAULT TRUE,  
7   failed_login_attempts INT DEFAULT 0,  
8   locked_until DATETIME NULL  
9 );  
10  
11 CREATE TABLE Student_Enrollment (  
12   id VARCHAR(36) PRIMARY KEY,  
13   enrollment_status ENUM('approving', 'pending', 'declining') DEFAULT 'pending',  
14   student_id VARCHAR(36) NOT NULL,  
15   course_section_id VARCHAR(36) NOT NULL,  
16   comment TEXT NULL,  
17   FOREIGN KEY (student_id) REFERENCES Student(id) ON DELETE CASCADE,  
18   FOREIGN KEY (course_section_id) REFERENCES Course_Section(id) ON  
19   DELETE CASCADE  
20 );
```

Listing 4.1: Core Database Schema Example

4.3 Database Normalization (3NF)

The schema was modeled in Third Normal Form (3NF) to eliminate redundancies:

- **1NF:** All attributes contain atomic values.
- **2NF:** No partial dependencies exist. All non-key fields depend entirely on the composite primary keys (e.g., student ID + course section ID in the enrollment table).
- **3NF:** No transitive dependencies are present. Student attributes (like advisor) are housed strictly in the Student table, while advisor department information resides in the Staff table.

4.4 Indexing Strategy

To guarantee efficient data retrieval, the following indexes are defined:

- Unique index on `Users.email` to support rapid lookup during authentication.
- Composite index on `Student_Enrollment(student_id, course_section_id)` to accelerate enrollment validation and student dashboard renders.

Chapter 5

Implementation Details

5.1 Key Algorithms and Business Logic

5.1.1 Schedule Conflict Detection

Before a student is permitted to request enrollment in a course section, the service compares the requested section's schedules against the student's existing active drafts and approved enrollments for that term.

```
1 for (const newSched of section.Schedule) {
2   const newStart = dateToMinutes(newSched.start_time);
3   const newEnd = dateToMinutes(newSched.end_time);
4
5   for (const oldSched of existingSchedules) {
6     if (newSched.day_of_week === oldSched.day_of_week) {
7       const oldStart = dateToMinutes(oldSched.start_time);
8       const oldEnd = dateToMinutes(oldSched.end_time);
9
10      if (newStart < oldEnd && newEnd > oldStart) {
11        throw new ApiError(409, 'Schedule conflict on ${newSched.
12          day_of_week}');
13      }
14    }
15  }
```

Listing 5.1: Schedule Overlap Detection

5.2 Security Implementation

- **Password Hashing:** Implemented via BCrypt. All plaintext passwords submitted during signup or reset are hashed with 10 salt rounds before database storage.
- **XSS Prevention:** A global sanitize middleware ([sanitizeBody.ts](file:///Users/dylantran/Documents/management-system/backend/src/middlewares/sanitizeBody.ts)) inspects and cleans incoming request payloads recursively, removing script tags.

5.3 Error Handling Strategy

We implement a global Express error-handling middleware that intercepts thrown errors. Expected errors are encapsulated using a custom 'ApiError' class containing status codes. Unexpected runtime exceptions are caught, logged in the console, and hidden behind a generic '500 Internal Server Error' response to prevent revealing sensitive stack traces.

5.4 AI Assistance Report

5.4.1 AI Assistance Summary

During the construction of the School Management System, AI tools were leveraged to improve coding speed and code safety:

- **Claude:** Utilized for core database schemas, validation middleware debugging, and complex SQL transaction lock checks (approximately 45% of structural design tasks).
- **GitHub Copilot:** Assisted in writing repetitive boilerplate components, react hooks, and layout styling mappings (approximately 35% of frontend development).

5.4.2 Major Components Where AI Was Used

Request Validation Middleware

During validation middleware testing, we ran into an issue where 'req.params' was being overwritten as 'undefined' when using schemas that only validated the request body. Claude identified that because the schema did not explicitly validate the request params, the parsed object returned by Zod stripped them. Based on this analysis, we modified the middleware logic to perform conditional checks before overriding the request properties:

```
1 if (parsedData.body !== undefined) req.body = parsedData.body;  
2 if (parsedData.query !== undefined) req.query = parsedData.query;  
3 if (parsedData.params !== undefined) req.params = parsedData.params;
```

Listing 5.2: validateRequest Correction

Chapter 6

Testing and Quality Assurance

6.1 Testing Strategy

The QA process combined automated type-safety checking with functional validation. We compiled both components without emit flags to confirm complete TypeScript static safety:

```
1 pnpm --filter backend exec tsc --noEmit
2 pnpm --filter frontend exec tsc --noEmit
```

6.2 Test Scenarios

Fifteen test cases were executed to verify core business logic:

ID	Target Scenario	Expected Result	Status
TC-01	Overdue invoice payment	Block enrollment attempt	Pass
TC-02	Request with invalid credentials	Block session token generation	Pass
TC-03	Registration over credit cap	Deny database write	Pass
TC-04	Schedule conflict attempt	Raise overlap conflict	Pass
TC-05	Duplicate registration check	Block second section addition	Pass

Table 6.1: Selected Verification Scenarios

6.3 Known Limitations

- **Cache Invalidation:** High-latency networks may experience a short delay in updating calendar slots due to cache delays in query caching.
- **Interactive Notifications:** Real-time notification updates when an advisor approves a student's request are pending WebSocket integration.

Chapter 7

Deployment

7.1 Deployment Process

7.1.1 Frontend Hosting

The React application is hosted on Vercel. Continuous deployment is configured by linking the GitHub repository, building with the command ‘pnpm build’, and outputting the directory ‘dist’ to Vercel’s global CDN nodes.

7.1.2 Backend Hosting

The Node.js Express server is deployed to Railway. A MySQL service instance runs concurrently on Railway with custom connection parameters exposed via environment variables.

7.2 Environment Variables

To run in production, the application configures:

- DATABASE_URL - Relational MySQL connection string.
- JWT_SECRET - Private token signature key.
- CORS_ORIGIN - Permitted frontend URL origins.

Chapter 8

User Guide

This chapter provides a detailed, step-by-step walkthrough of the Edusoft School Management System. It covers user authentication, course registration workflows (including the search feature), timetable visualization for students, and the registration approval interface for academic advisors.

8.1 Getting Started and Authentication

To access the Edusoft portal, navigate to the frontend client URL: <https://school-management-system-hcmiu.vercel.app/>. Users must authenticate using their institutional credentials to access their role-specific workspaces.

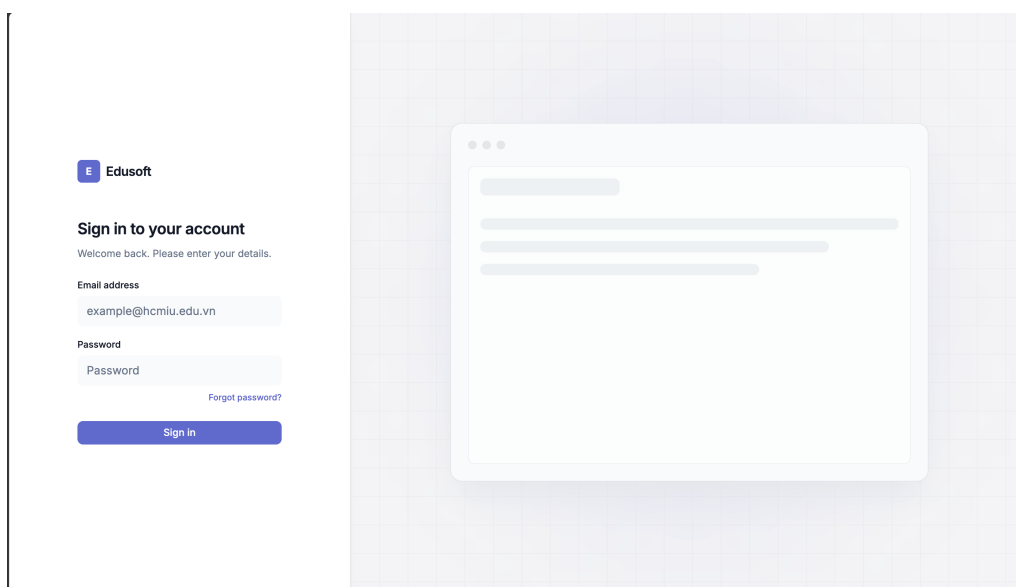


Figure 8.1: Edusoft Portal Authentication Screen

As shown in Figure 8.1, the login interface features a sleek side-by-side layout:

1. Enter the institutional email address in the **Email address** field (e.g., `ITITWE24025@student.hcm` for students or `nhphu@hcmiu.edu.vn` for advisors).
2. Input the account password in the **Password** field (e.g., `Password123!`).
3. Click the purple **Sign in** button to authenticate.

8.2 Course Registration and Searching

After authenticating as a student, select **Register Courses** from the left navigation sidebar to open the course enrollment dashboard.

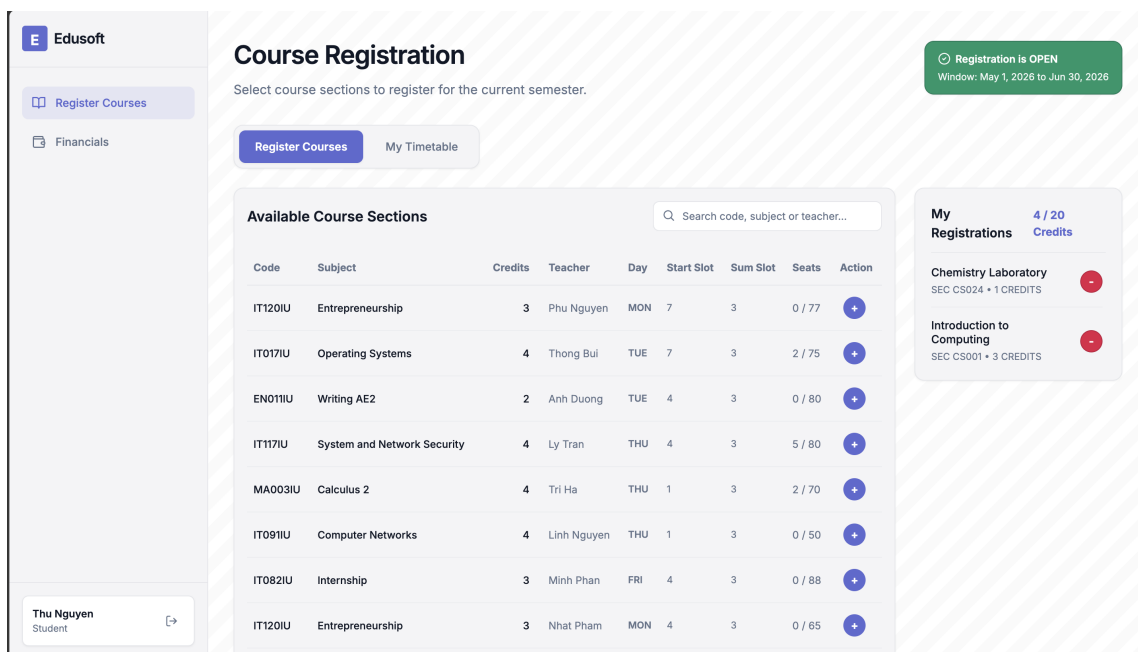


Figure 8.2: Course Registration Panel with Course Offerings and Selected Drafts

As shown in Figure 8.2, the student's registration interface is divided into key regions:

- **Status Indicator:** The top-right badge indicates whether the registration window is currently active (e.g., "Registration is OPEN" from May 1, 2026 to June 30, 2026).
- **Search Bar:** Located directly above the available sections table, the search box filters courses instantly by subject code, subject title, or teacher name. For example, typing a course code will immediately hide unrelated rows.
- **Available Course Sections:** This table lists all active courses with details including credit counts, schedules, and remaining seats. Students can request registration by clicking the blue + button in the **Action** column.
- **My Registrations Sidebar:** The sidebar on the right tracks requested courses in real-time. It displays total registered credits against the allowed limit (e.g., 4 / 20 Credits) and provides a red - button to remove sections from the draft.

8.3 Weekly Class Schedule Timetable

To view your active weekly schedule, navigate to the registration workspace and click the **My Timetable** tab toggle.

As displayed in Figure 8.3, the timetable maps all approved course sections onto a clean calendar grid:

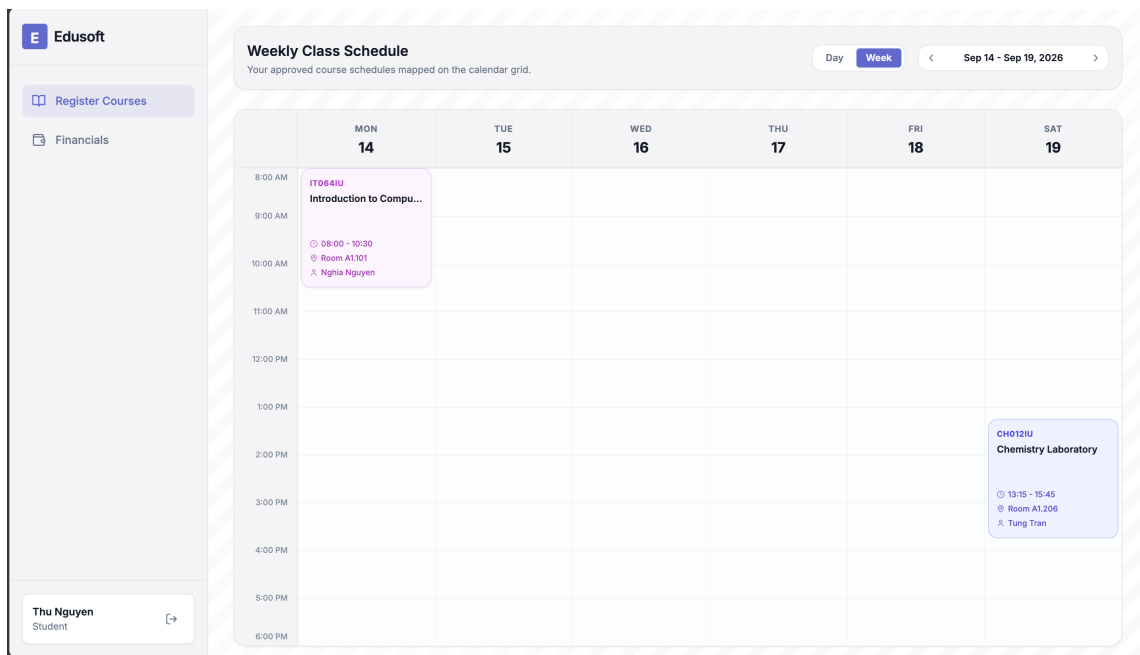


Figure 8.3: Personal Weekly Class Schedule Timetable Grid

- The grid spans from Monday to Saturday, broken down hourly from 8:00 AM to 6:00 PM.
- Each approved course card shows the course code, subject name, class hours, class-room, and instructor.
- Toggles at the top-right allow switching between Day and Week views, and navigating across different calendar weeks.

8.4 Advising Approval Walkthrough

For academic advisors (Staff), the student advising request workflow operates as follows:

1. Log in as an advisor and select **Advisor Approvals** in the sidebar menu.
2. Review advising cards grouped by student. Each card displays the list of sections the student requested.
3. Click **Approve All** to approve all selected courses, or input a rejection reason comment and click **Decline** for individual course sections.

Chapter 9

Team Collaboration

9.1 Team Organization

Our team divided work based on frontend and backend boundaries:

- **Tran Anh Van:** Lead backend architecture, relational database configuration, security policies, and deployment configurations.
- **Nguyen Tran Minh Thu:** Lead frontend user interface development, state synchronization, schedule calendar views, and pagination components.

9.2 Individual Contributions

- **Tran Anh Van (50%):** Implemented Express routes, database migrations, conflict-check service transactions, and CORS setup.
- **Nguyen Tran Minh Thu (50%):** Crafted Tailwind components, React Query data integrations, login authentication forms, and responsive advisor dashboards.

Chapter 10

Conclusion and Reflection

10.1 Project Summary

The Edusoft School Management System successfully achieves all base and advanced targets specified in the curriculum user stories. We engineered a scalable, responsive, and robust academic registry tool that preserves transaction safety and processes registrations safely.

10.2 Lessons Learned

- **Validation Synchronization:** Keeping database column constraints, Zod request schemas, and frontend validation models synchronous requires strict API contracts.
- **Transaction Locking:** Database locks like ‘FOR UPDATE’ are vital for concurrent slots validation to avoid dirty reads in high-throughput academic registration events.

Appendix A

Meeting Logs

The team held weekly meetings to coordinate development and review implementation progress:

- **Meeting 1 (Week 1):** Project kick-off. Defined scope, user story domains, and set up git repositories. Designed the initial relational database schema.
- **Meeting 2 (Week 2):** Backend setup and User onboarding module review. Implemented session validation, BCrypt password hashing, and user role validation rules.
- **Meeting 3 (Week 3):** Frontend UI styling and React Query integration. Set up Vite, configured routing paths, and finalized student course selection panels.
- **Meeting 4 (Week 4):** Academic advising workflows and Tuition ledger calculation logic. Implemented advisors' approvals grouped interface and dynamic invoice recalculation checks.
- **Meeting 5 (Week 5):** System settings and calendar bounds. Deployed the backend to Railway and the frontend client to Vercel. Finalized end-to-end user testing.

Appendix B

API Documentation (OpenAPI 3.1.0 Specification)

Below is the complete API specification in OpenAPI 3.1.0 format, loaded from `openapi.yaml`:

```
1 openapi: '3.1.0'
2 info:
3   title: School Management System API
4   version: '1.0.0'
5   description: API specification for a School Management System with
6     user authentication, academic management, scheduling, fees, and
7     payroll processing.
8 servers:
9   - url: https://api.example.com/api/v1
10 tags:
11   - name: auth
12     description: Authentication operations
13   - name: users
14     description: User account management with roles
15   - name: staff
16     description: Staff profile management
17   - name: students
18     description: Student profile management
19   - name: subjects
20     description: Academic subjects management
21   - name: courseSections
22     description: Course sections linking subjects, staff, and semesters
23   - name: schedules
24     description: Scheduling of course sections in rooms and times
25   - name: fees
26     description: Student fee tracking and payments
27   - name: enrollments
28     description: Student enrollments in course sections
29   - name: salaries
30     description: Payroll processing for staff
31   - name: rooms
32     description: Room management
33 paths:
34   /auth/login:
35     post:
36       tags:
37         - auth
38       summary: User login
39       description: Authenticate user and return JWT token.
```

```
38     requestBody:
39       required: true
40       content:
41         application/json:
42           schema:
43             type: object
44             required:
45               - email
46               - password
47             properties:
48               email:
49                 type: string
50                 format: email
51               password:
52                 type: string
53                 format: password
54     responses:
55       '200':
56         description: Successful login with JWT token
57         headers:
58           Set-Cookie:
59             description: Contains the HttpOnly, Secure JWT cookie.
60             schema:
61               type: string
62             content: {}
63       '400':
64         description: Bad request (missing or invalid credentials)
65       '401':
66         description: Unauthorized (invalid username or password)
67       '403':
68         description: Forbidden (Account is inactive or suspended).
69       '429':
70         description: Too Many Requests (Account temporarily locked
71         due to excessive failed attempts).
72 /auth/logout:
73   post:
74     tags:
75       - auth
76     summary: User logout
77     description: Invalidate user session or token.
78     security:
79       - cookieAuth: []
80     responses:
81       '200':
82         description: Successful logout
83         headers:
84           Set-Cookie:
85             description: Clears the JWT cookie by setting Max-Age=0.
86             schema:
87               type: string
88       '401':
89         description: Unauthorized (invalid or missing token)
90 /users:
91   get:
92     tags:
93       - users
94     summary: Get all users
95     description: Requires Admin Role.
```

```

95     security:
96       - cookieAuth: []
97   responses:
98     '200':
99       description: List of users
100      content:
101        application/json:
102          schema:
103            type: array
104            items:
105              $ref: '#/components/schemas/User'
106     '401':
107       description: Unauthorized
108     '403':
109       description: Forbidden (requires Admin role)
110  post:
111    tags:
112      - users
113    summary: Create a new user
114    description: Requires Admin Role.
115    security:
116      - cookieAuth: []
117    requestBody:
118      required: true
119      content:
120        application/json:
121          schema:
122            $ref: '#/components/schemas/UserCreate'
123    responses:
124      '201':
125        description: User created successfully
126        content:
127          application/json:
128            schema:
129              $ref: '#/components/schemas/User'
130      '400':
131        description: Bad request
132      '401':
133        description: Unauthorized
134      '403':
135        description: Forbidden (requires Admin role)
136      '409':
137        description: Conflict (email already exists in the system)
138  /users/{userId}:
139    get:
140      tags:
141        - users
142      summary: Get user by ID
143      security:
144        - cookieAuth: []
145      parameters:
146        - name: userId
147          in: path
148          required: true
149          schema:
150            type: string
151      responses:
152        '200':

```

```

153         description: User details
154         content:
155             application/json:
156                 schema:
157                     $ref: '#/components/schemas/User'
158     '400':
159         description: Bad request
160     '401':
161         description: Unauthorized
162     '404':
163         description: User not found
164 put:
165     tags:
166         - users
167     summary: Update user by ID
168     description: Requires Admin Role.
169     security:
170         - cookieAuth: []
171     parameters:
172         - name: userId
173           in: path
174           required: true
175           schema:
176               type: string
177     requestBody:
178         required: true
179         content:
180             application/json:
181                 schema:
182                     $ref: '#/components/schemas/UserUpdate'
183     responses:
184         '200':
185             description: User updated successfully
186         '400':
187             description: Bad request
188         '401':
189             description: Unauthorized
190         '403':
191             description: Forbidden (requires Admin role)
192         '404':
193             description: User not found
194         '409':
195             description: Conflict (email already exists in the system)
196 delete:
197     tags:
198         - users
199     summary: Delete user by ID
200     description: Requires Admin Role.
201     security:
202         - cookieAuth: []
203     parameters:
204         - name: userId
205           in: path
206           required: true
207           schema:
208               type: string
209     responses:
210         '200':

```

```

211         description: User deleted successfully
212     '400':
213         description: Bad request
214     '401':
215         description: Unauthorized
216     '403':
217         description: Forbidden (requires Admin role)
218     '404':
219         description: User not found
220 /staff:
221     get:
222         tags:
223             - staff
224         summary: Get all staff members
225         security:
226             - cookieAuth: []
227         responses:
228             '200':
229                 description: List of staff members
230                 content:
231                     application/json:
232                         schema:
233                             type: array
234                             items:
235                                 $ref: '#/components/schemas/Staff'
236             '401':
237                 description: Unauthorized
238     post:
239         tags:
240             - staff
241         summary: Create a new staff member
242         security:
243             - cookieAuth: []
244         requestBody:
245             required: true
246             content:
247                 application/json:
248                     schema:
249                         $ref: '#/components/schemas/StaffCreate'
250         responses:
251             '201':
252                 description: Staff member created successfully
253                 content:
254                     application/json:
255                         schema:
256                             $ref: '#/components/schemas/Staff'
257             '400':
258                 description: Bad request
259             '401':
260                 description: Unauthorized
261             '409':
262                 description: Conflict (employee_code already exists)
263 /staff/{staffId}:
264     get:
265         tags:
266             - staff
267         summary: Get staff member by ID
268         security:

```

```
269     - cookieAuth: []
270 parameters:
271     - name: staffId
272       in: path
273       required: true
274       schema:
275         type: string
276 responses:
277     '200':
278       description: Staff member details
279       content:
280         application/json:
281           schema:
282             $ref: '#/components/schemas/Staff'
283     '400':
284       description: Bad request
285     '401':
286       description: Unauthorized
287     '404':
288       description: Staff member not found
289 put:
290   tags:
291     - staff
292   summary: Update staff member by ID
293   security:
294     - cookieAuth: []
295   parameters:
296     - name: staffId
297       in: path
298       required: true
299       schema:
300         type: string
301   requestBody:
302     required: true
303     content:
304       application/json:
305         schema:
306           $ref: '#/components/schemas/StaffUpdate'
307   responses:
308     '200':
309       description: Staff member updated successfully
310     '400':
311       description: Bad request
312     '401':
313       description: Unauthorized
314     '404':
315       description: Staff member not found
316     '409':
317       description: Conflict (employee_code already exists)
318 delete:
319   tags:
320     - staff
321   summary: Delete staff member by ID
322   security:
323     - cookieAuth: []
324   parameters:
325     - name: staffId
326       in: path
```

```

327         required: true
328         schema:
329             type: string
330     responses:
331         '200':
332             description: Staff member deleted successfully
333         '400':
334             description: Bad request
335         '401':
336             description: Unauthorized
337         '404':
338             description: Staff member not found
339 /staff/{staffId}/salaries:
340     get:
341         tags:
342             - salaries
343         summary: Get salary history for a specific staff member
344         description: Allows staff members to view their own payslips.
345         security:
346             - cookieAuth: []
347         parameters:
348             - name: staffId
349               in: path
350               required: true
351               schema:
352                   type: string
353                   format: uuid
354         responses:
355             '200':
356                 description: Salary history for staff member
357                 content:
358                     application/json:
359                         schema:
360                             type: array
361                             items:
362                                 $ref: '#/components/schemas/SalaryResult'
363             '400':
364                 description: Bad request
365             '401':
366                 description: Unauthorized
367             '404':
368                 description: Staff member not found
369 /students:
370     get:
371         tags:
372             - students
373         summary: Get all students
374         security:
375             - cookieAuth: []
376         responses:
377             '200':
378                 description: List of students
379                 content:
380                     application/json:
381                         schema:
382                             type: array
383                             items:
384                                 $ref: '#/components/schemas/Student'

```

```

385         '401':
386             description: Unauthorized
387 post:
388     tags:
389         - students
390     summary: Create a new student
391     security:
392         - cookieAuth: []
393     requestBody:
394         required: true
395         content:
396             application/json:
397                 schema:
398                     $ref: '#/components/schemas/StudentCreate'
399 responses:
400     '201':
401         description: Student created successfully
402         content:
403             application/json:
404                 schema:
405                     $ref: '#/components/schemas/Student'
406     '400':
407         description: Bad request
408     '401':
409         description: Unauthorized
410     '409':
411         description: Conflict (student_code already exists)
412 /students/{studentId}:
413 get:
414     tags:
415         - students
416     summary: Get student by ID
417     security:
418         - cookieAuth: []
419     parameters:
420         - name: studentId
421           in: path
422           required: true
423           schema:
424               type: string
425     responses:
426     '200':
427         description: Student details
428         content:
429             application/json:
430                 schema:
431                     $ref: '#/components/schemas/Student'
432     '400':
433         description: Bad request
434     '401':
435         description: Unauthorized
436     '404':
437         description: Student not found
438 put:
439     tags:
440         - students
441     summary: Update student by ID
442     security:

```

```

443     - cookieAuth: []
444 parameters:
445     - name: studentId
446       in: path
447       required: true
448       schema:
449         type: string
450 requestBody:
451     required: true
452     content:
453       application/json:
454         schema:
455           $ref: '#/components/schemas/StudentUpdate'
456 responses:
457   '200':
458     description: Student updated successfully
459   '400':
460     description: Bad request
461   '401':
462     description: Unauthorized
463   '404':
464     description: Student not found
465   '409':
466     description: Conflict (student_code already exists)
467 delete:
468   tags:
469     - students
470   summary: Delete student by ID
471   security:
472     - cookieAuth: []
473   parameters:
474     - name: studentId
475       in: path
476       required: true
477       schema:
478         type: string
479   responses:
480     '200':
481       description: Student deleted successfully
482     '400':
483       description: Bad request
484     '401':
485       description: Unauthorized
486     '404':
487       description: Student not found
488 /students/{studentId}/timetable:
489 get:
490   tags:
491     - students
492   summary: Get timetable for a student
493   description: Returns joined enrollments with schedules for the
student.
494   security:
495     - cookieAuth: []
496   parameters:
497     - name: studentId
498       in: path
499       required: true

```

```

500         schema:
501             type: string
502     responses:
503         '200':
504             description: Student timetable
505             content:
506                 application/json:
507                     schema:
508                         type: array
509                         items:
510                             $ref: '#/components/schemas/Schedule'
511         '400':
512             description: Bad request
513         '401':
514             description: Unauthorized
515         '404':
516             description: Student not found
517 /subjects:
518     get:
519         tags:
520             - subjects
521         summary: Get all subjects
522         security:
523             - cookieAuth: []
524         responses:
525             '200':
526                 description: List of subjects
527                 content:
528                     application/json:
529                         schema:
530                             type: array
531                             items:
532                                 $ref: '#/components/schemas/Subject'
533             '401':
534                 description: Unauthorized
535     post:
536         tags:
537             - subjects
538         summary: Create a new subject
539         security:
540             - cookieAuth: []
541         requestBody:
542             required: true
543             content:
544                 application/json:
545                     schema:
546                         $ref: '#/components/schemas/SubjectCreate'
547         responses:
548             '201':
549                 description: Subject created successfully
550                 content:
551                     application/json:
552                         schema:
553                             $ref: '#/components/schemas/Subject'
554             '400':
555                 description: Bad request
556             '401':
557                 description: Unauthorized

```

```
558         '409':
559             description: Conflict (subject code already exists)
560 /subjects/{subjectId}:
561     get:
562         tags:
563             - subjects
564         summary: Get subject by ID
565         security:
566             - cookieAuth: []
567         parameters:
568             - name: subjectId
569               in: path
570               required: true
571               schema:
572                 type: string
573         responses:
574             '200':
575                 description: Subject details
576                 content:
577                     application/json:
578                         schema:
579                             $ref: '#/components/schemas/Subject'
580             '400':
581                 description: Bad request
582             '401':
583                 description: Unauthorized
584             '404':
585                 description: Subject not found
586     put:
587         tags:
588             - subjects
589         summary: Update subject by ID
590         security:
591             - cookieAuth: []
592         parameters:
593             - name: subjectId
594               in: path
595               required: true
596               schema:
597                 type: string
598         requestBody:
599             required: true
600             content:
601                 application/json:
602                     schema:
603                         $ref: '#/components/schemas/SubjectUpdate'
604         responses:
605             '200':
606                 description: Subject updated successfully
607             '400':
608                 description: Bad request
609             '401':
610                 description: Unauthorized
611             '404':
612                 description: Subject not found
613             '409':
614                 description: Conflict (subject code already exists)
615     delete:
```

```
616     tags:
617         - subjects
618     summary: Delete subject by ID
619     security:
620         - cookieAuth: []
621     parameters:
622         - name: subjectId
623           in: path
624           required: true
625           schema:
626             type: string
627     responses:
628         '200':
629             description: Subject deleted successfully
630         '400':
631             description: Bad request
632         '401':
633             description: Unauthorized
634         '404':
635             description: Subject not found
636 /courseSections:
637     get:
638         tags:
639             - courseSections
640         summary: Get all course sections
641         security:
642             - cookieAuth: []
643         responses:
644             '200':
645                 description: List of course sections
646                 content:
647                     application/json:
648                         schema:
649                             type: array
650                             items:
651                                 $ref: '#/components/schemas/CourseSection'
652             '401':
653                 description: Unauthorized
654     post:
655         tags:
656             - courseSections
657         summary: Create a new course section
658         description: Requires Admin Role.
659         security:
660             - cookieAuth: []
661         requestBody:
662             required: true
663             content:
664                 application/json:
665                     schema:
666                         $ref: '#/components/schemas/CourseSectionCreate'
667         responses:
668             '201':
669                 description: Course section created successfully
670                 content:
671                     application/json:
672                         schema:
673                             $ref: '#/components/schemas/CourseSection'
```

```
674     '400':
675         description: Bad request
676     '401':
677         description: Unauthorized
678     '403':
679         description: Forbidden (requires Admin role)
680 /courseSections/{sectionId}:
681 get:
682     tags:
683         - courseSections
684     summary: Get course section by ID
685     security:
686         - cookieAuth: []
687     parameters:
688         - name: sectionId
689           in: path
690           required: true
691           schema:
692             type: string
693     responses:
694         '200':
695             description: Course section details
696             content:
697                 application/json:
698                     schema:
699                         $ref: '#/components/schemas/CourseSection'
700         '400':
701             description: Bad request
702         '401':
703             description: Unauthorized
704         '404':
705             description: Course section not found
706 put:
707     tags:
708         - courseSections
709     summary: Update course section by ID
710     description: Requires Admin Role.
711     security:
712         - cookieAuth: []
713     parameters:
714         - name: sectionId
715           in: path
716           required: true
717           schema:
718             type: string
719     requestBody:
720         required: true
721         content:
722             application/json:
723                 schema:
724                     $ref: '#/components/schemas/CourseSectionUpdate'
725     responses:
726         '200':
727             description: Course section updated successfully
728         '400':
729             description: Bad request
730         '401':
731             description: Unauthorized
```

```
732     '403':
733         description: Forbidden (requires Admin role)
734     '404':
735         description: Course section not found
736 delete:
737     tags:
738         - courseSections
739     summary: Delete course section by ID
740     description: Requires Admin Role.
741     security:
742         - cookieAuth: []
743     parameters:
744         - name: sectionId
745           in: path
746           required: true
747           schema:
748               type: string
749     responses:
750         '200':
751             description: Course section deleted successfully
752         '400':
753             description: Bad request
754         '401':
755             description: Unauthorized
756         '403':
757             description: Forbidden (requires Admin role)
758         '404':
759             description: Course section not found
760 /schedules:
761     get:
762         tags:
763             - schedules
764         summary: Get all schedules
765         security:
766             - cookieAuth: []
767         responses:
768             '200':
769                 description: List of schedules
770                 content:
771                     application/json:
772                         schema:
773                             type: array
774                             items:
775                                 $ref: '#/components/schemas/Schedule'
776             '401':
777                 description: Unauthorized
778     post:
779         tags:
780             - schedules
781         summary: Create a new schedule
782         description: Requires Admin Role.
783         security:
784             - cookieAuth: []
785         requestBody:
786             required: true
787             content:
788                 application/json:
789                     schema:
```

```

790         $ref: '#/components/schemas/ScheduleCreate'
791     responses:
792         '201':
793             description: Schedule created successfully
794             content:
795                 application/json:
796                     schema:
797                         $ref: '#/components/schemas/Schedule'
798         '400':
799             description: Bad request
800         '401':
801             description: Unauthorized
802         '403':
803             description: Forbidden (requires Admin role)
804         '409':
805             description: Conflict (room scheduling clash)
806 /schedules/{scheduleId}:
807     get:
808         tags:
809             - schedules
810         summary: Get schedule by ID
811         security:
812             - cookieAuth: []
813         parameters:
814             - name: scheduleId
815               in: path
816               required: true
817               schema:
818                 type: string
819         responses:
820             '200':
821                 description: Schedule details
822                 content:
823                     application/json:
824                         schema:
825                             $ref: '#/components/schemas/Schedule'
826             '400':
827                 description: Bad request
828             '401':
829                 description: Unauthorized
830             '404':
831                 description: Schedule not found
832     put:
833         tags:
834             - schedules
835         summary: Update schedule by ID
836         description: Requires Admin Role.
837         security:
838             - cookieAuth: []
839         parameters:
840             - name: scheduleId
841               in: path
842               required: true
843               schema:
844                 type: string
845         requestBody:
846             required: true
847             content:

```

```

848         application/json:
849             schema:
850                 $ref: '#/components/schemas/ScheduleUpdate'
851     responses:
852         '200':
853             description: Schedule updated successfully
854         '400':
855             description: Bad request
856         '401':
857             description: Unauthorized
858         '403':
859             description: Forbidden (requires Admin role)
860         '404':
861             description: Schedule not found
862         '409':
863             description: Conflict (room scheduling clash)
864     delete:
865         tags:
866             - schedules
867         summary: Delete schedule by ID
868         description: Requires Admin Role.
869         security:
870             - cookieAuth: []
871         parameters:
872             - name: scheduleId
873               in: path
874               required: true
875               schema:
876                 type: string
877         responses:
878             '200':
879                 description: Schedule deleted successfully
880             '400':
881                 description: Bad request
882             '401':
883                 description: Unauthorized
884             '403':
885                 description: Forbidden (requires Admin role)
886             '404':
887                 description: Schedule not found
888 /fees:
889     get:
890         tags:
891             - fees
892         summary: Get all fees records
893         security:
894             - cookieAuth: []
895         responses:
896             '200':
897                 description: List of fees records
898                 content:
899                     application/json:
900                         schema:
901                             type: array
902                             items:
903                                 $ref: '#/components/schemas/Fee'
904             '401':
905                 description: Unauthorized

```

```

906 post:
907   tags:
908     - fees
909   summary: Create a new fee record
910   security:
911     - cookieAuth: []
912   requestBody:
913     required: true
914     content:
915       application/json:
916         schema:
917           $ref: '#/components/schemas/FeeCreate'
918   responses:
919     '201':
920       description: Fee record created successfully
921       content:
922         application/json:
923           schema:
924             $ref: '#/components/schemas/Fee'
925     '400':
926       description: Bad request
927     '401':
928       description: Unauthorized
929 /fees/{feeId}:
930   get:
931     tags:
932       - fees
933     summary: Get fee record by ID
934     security:
935       - cookieAuth: []
936     parameters:
937       - name: feeId
938         in: path
939         required: true
940         schema:
941           type: string
942     responses:
943       '200':
944         description: Fee record details
945         content:
946           application/json:
947             schema:
948               $ref: '#/components/schemas/Fee'
949       '400':
950         description: Bad request
951       '401':
952         description: Unauthorized
953       '404':
954         description: Fee record not found
955   put:
956     tags:
957       - fees
958     summary: Update fee record by ID
959     security:
960       - cookieAuth: []
961     parameters:
962       - name: feeId
963         in: path

```

```

964         required: true
965         schema:
966             type: string
967     requestBody:
968         required: true
969         content:
970             application/json:
971                 schema:
972                     $ref: '#/components/schemas/FeeUpdate'
973     responses:
974         '200':
975             description: Fee record updated successfully
976         '400':
977             description: Bad request
978         '401':
979             description: Unauthorized
980         '404':
981             description: Fee record not found
982     delete:
983         tags:
984             - fees
985         summary: Delete fee record by ID
986         security:
987             - cookieAuth: []
988         parameters:
989             - name: feeId
990               in: path
991               required: true
992               schema:
993                   type: string
994         responses:
995             '200':
996                 description: Fee record deleted successfully
997             '400':
998                 description: Bad request
999             '401':
1000                 description: Unauthorized
1001             '404':
1002                 description: Fee record not found
1003 /enrollments:
1004     get:
1005         tags:
1006             - enrollments
1007         summary: Get all enrollments
1008         security:
1009             - cookieAuth: []
1010         responses:
1011             '200':
1012                 description: List of enrollments
1013                 content:
1014                     application/json:
1015                         schema:
1016                             type: array
1017                             items:
1018                                 $ref: '#/components/schemas/Enrollment'
1019             '401':
1020                 description: Unauthorized
1021     post:

```

```
1022     tags:
1023       - enrollments
1024     summary: Create a new enrollment
1025     description: Block enrollment if student has overdue fees.
1026     security:
1027       - cookieAuth: []
1028     requestBody:
1029       required: true
1030       content:
1031         application/json:
1032           schema:
1033             $ref: '#/components/schemas/EnrollmentCreate'
1034     responses:
1035       '201':
1036         description: Enrollment created successfully
1037         content:
1038           application/json:
1039             schema:
1040               $ref: '#/components/schemas/Enrollment'
1041       '400':
1042         description: Bad request
1043       '401':
1044         description: Unauthorized
1045       '403':
1046         description: Forbidden (student has overdue fees)
1047 /enrollments/{enrollmentId}:
1048   get:
1049     tags:
1050       - enrollments
1051     summary: Get enrollment by ID
1052     security:
1053       - cookieAuth: []
1054     parameters:
1055       - name: enrollmentId
1056         in: path
1057         required: true
1058         schema:
1059           type: string
1060     responses:
1061       '200':
1062         description: Enrollment details
1063         content:
1064           application/json:
1065             schema:
1066               $ref: '#/components/schemas/Enrollment'
1067       '400':
1068         description: Bad request
1069       '401':
1070         description: Unauthorized
1071       '404':
1072         description: Enrollment not found
1073   put:
1074     tags:
1075       - enrollments
1076     summary: Update enrollment by ID
1077     security:
1078       - cookieAuth: []
1079     parameters:
```

```

1080     - name: enrollmentId
1081       in: path
1082       required: true
1083       schema:
1084         type: string
1085 requestBody:
1086   required: true
1087   content:
1088     application/json:
1089       schema:
1090         $ref: '#/components/schemas/EnrollmentUpdate'
1091 responses:
1092   '200':
1093     description: Enrollment updated successfully
1094   '400':
1095     description: Bad request
1096   '401':
1097     description: Unauthorized
1098   '403':
1099     description: Forbidden (student has overdue fees)
1100   '404':
1101     description: Enrollment not found
1102 delete:
1103   tags:
1104     - enrollments
1105   summary: Delete enrollment by ID
1106   security:
1107     - cookieAuth: []
1108   parameters:
1109     - name: enrollmentId
1110       in: path
1111       required: true
1112       schema:
1113         type: string
1114   responses:
1115     '200':
1116       description: Enrollment deleted successfully
1117     '400':
1118       description: Bad request
1119     '401':
1120       description: Unauthorized
1121     '404':
1122       description: Enrollment not found
1123 /salaries:
1124   get:
1125     tags:
1126       - salaries
1127     summary: Get all processed salaries
1128     description: Admin endpoint to view all historical payroll
1129     records.
1130     security:
1131       - cookieAuth: []
1132   responses:
1133     '200':
1134       description: List of processed salaries
1135       content:
1136         application/json:
1137           schema:

```

```

1137         type: array
1138         items:
1139             $ref: '#/components/schemas/SalaryResult'
1140     '401':
1141         description: Unauthorized
1142     '403':
1143         description: Forbidden
1144 post:
1145     tags:
1146         - salaries
1147     summary: Process payroll
1148     description: Admin endpoint to process payroll (Base + Allowances
1149 - Deductions).
1149     security:
1150         - cookieAuth: []
1151     requestBody:
1152         required: true
1153         content:
1154             application/json:
1155                 schema:
1156                     $ref: '#/components/schemas/SalaryProcess'
1157     responses:
1158         '200':
1159             description: Payroll processed successfully
1160             content:
1161                 application/json:
1162                     schema:
1163                         $ref: '#/components/schemas/SalaryResult'
1164         '400':
1165             description: Bad request
1166         '401':
1167             description: Unauthorized
1168         '403':
1169             description: Forbidden (non-admin user)
1170 /rooms:
1171 get:
1172     tags:
1173         - rooms
1174     summary: Get all rooms
1175     security:
1176         - cookieAuth: []
1177     responses:
1178         '200':
1179             description: List of rooms
1180             content:
1181                 application/json:
1182                     schema:
1183                         type: array
1184                         items:
1185                             $ref: '#/components/schemas/Room'
1186         '401':
1187             description: Unauthorized
1188 post:
1189     tags:
1190         - rooms
1191     summary: Create a new room
1192     security:
1193         - cookieAuth: []

```

```
1194     requestBody:
1195       required: true
1196       content:
1197         application/json:
1198           schema:
1199             $ref: '#/components/schemas/RoomCreate'
1200 responses:
1201   '201':
1202     description: Room created successfully
1203     content:
1204       application/json:
1205         schema:
1206           $ref: '#/components/schemas/Room'
1207   '400':
1208     description: Bad request
1209   '401':
1210     description: Unauthorized
1211 /rooms/{roomId}:
1212 get:
1213   tags:
1214     - rooms
1215   summary: Get room by ID
1216   security:
1217     - cookieAuth: []
1218   parameters:
1219     - name: roomId
1220       in: path
1221       required: true
1222       schema:
1223         type: string
1224   responses:
1225     '200':
1226       description: Room details
1227       content:
1228         application/json:
1229           schema:
1230             $ref: '#/components/schemas/Room'
1231     '400':
1232       description: Bad request
1233     '401':
1234       description: Unauthorized
1235     '404':
1236       description: Room not found
1237 put:
1238   tags:
1239     - rooms
1240   summary: Update room by ID
1241   security:
1242     - cookieAuth: []
1243   parameters:
1244     - name: roomId
1245       in: path
1246       required: true
1247       schema:
1248         type: string
1249   requestBody:
1250     required: true
1251     content:
```

```
1252     application/json:
1253       schema:
1254         $ref: '#/components/schemas/RoomUpdate'
1255   responses:
1256     '200':
1257       description: Room updated successfully
1258     '400':
1259       description: Bad request
1260     '401':
1261       description: Unauthorized
1262     '404':
1263       description: Room not found
1264   delete:
1265     tags:
1266       - rooms
1267     summary: Delete room by ID
1268     security:
1269       - cookieAuth: []
1270     parameters:
1271       - name: roomId
1272         in: path
1273         required: true
1274         schema:
1275           type: string
1276     responses:
1277       '200':
1278         description: Room deleted successfully
1279       '400':
1280         description: Bad request
1281       '401':
1282         description: Unauthorized
1283       '404':
1284         description: Room not found
1285   components:
1286     securitySchemes:
1287       cookieAuth:
1288         type: apiKey
1289         in: cookie
1290         name: token
1291     schemas:
1292       User:
1293         type: object
1294         properties:
1295           id:
1296             type: string
1297             format: uuid
1298           email:
1299             type: string
1300             format: email
1301           role:
1302             type: string
1303             enum:
1304               - Admin
1305               - Staff
1306               - Student
1307         required:
1308           - id
1309           - email
```

```
1310     - role
1311   UserCreate:
1312     type: object
1313     required:
1314       - email
1315       - password
1316       - role
1317     properties:
1318       email:
1319         type: string
1320         format: email
1321       password:
1322         type: string
1323         format: password
1324       role:
1325         type: string
1326         enum:
1327           - Admin
1328           - Staff
1329           - Student
1330   UserUpdate:
1331     type: object
1332     properties:
1333       email:
1334         type: string
1335         format: email
1336       password:
1337         type: string
1338         format: password
1339       role:
1340         type: string
1341         enum:
1342           - Admin
1343           - Staff
1344           - Student
1345   Staff:
1346     type: object
1347     properties:
1348       id:
1349         type: string
1350         format: uuid
1351       employee_code:
1352         type: string
1353       department:
1354         type: string
1355       user:
1356         $ref: '#/components/schemas/User'
1357     required:
1358       - id
1359       - employee_code
1360       - department
1361       - user
1362   StaffCreate:
1363     type: object
1364     required:
1365       - employee_code
1366       - department
1367       - user
```

```
1368     properties:
1369         employee_code:
1370             type: string
1371         department:
1372             type: string
1373         user:
1374             $ref: '#/components/schemas/UserCreate'
1375 StaffUpdate:
1376     type: object
1377     properties:
1378         employee_code:
1379             type: string
1380         department:
1381             type: string
1382         user:
1383             $ref: '#/components/schemas/UserUpdate'
1384 Student:
1385     type: object
1386     properties:
1387         id:
1388             type: string
1389             format: uuid
1390         student_code:
1391             type: string
1392         user:
1393             $ref: '#/components/schemas/User'
1394     required:
1395         - id
1396         - student_code
1397         - user
1398 StudentCreate:
1399     type: object
1400     required:
1401         - student_code
1402         - user
1403     properties:
1404         student_code:
1405             type: string
1406         user:
1407             $ref: '#/components/schemas/UserCreate'
1408 StudentUpdate:
1409     type: object
1410     properties:
1411         student_code:
1412             type: string
1413         user:
1414             $ref: '#/components/schemas/UserUpdate'
1415 Subject:
1416     type: object
1417     properties:
1418         id:
1419             type: string
1420             format: uuid
1421         name:
1422             type: string
1423         code:
1424             type: string
1425         credits:
```

```
1426         type: integer
1427     required:
1428     - id
1429     - name
1430     - code
1431     - credits
1432 SubjectCreate:
1433     type: object
1434     required:
1435     - name
1436     - code
1437     - credits
1438     properties:
1439     name:
1440         type: string
1441     code:
1442         type: string
1443     credits:
1444         type: integer
1445 SubjectUpdate:
1446     type: object
1447     properties:
1448     name:
1449         type: string
1450     code:
1451         type: string
1452     credits:
1453         type: integer
1454 CourseSection:
1455     type: object
1456     properties:
1457     id:
1458         type: string
1459         format: uuid
1460     subject_id:
1461         type: string
1462         format: uuid
1463     staff_id:
1464         type: string
1465         format: uuid
1466     semester:
1467         type: string
1468     year:
1469         type: integer
1470     section_number:
1471         type: string
1472     required:
1473     - id
1474     - subject_id
1475     - staff_id
1476     - semester
1477     - year
1478     - section_number
1479 CourseSectionCreate:
1480     type: object
1481     required:
1482     - subject_id
1483     - staff_id
```

```
1484     - semester
1485     - year
1486     - section_number
1487   properties:
1488     subject_id:
1489       type: string
1490       format: uuid
1491     staff_id:
1492       type: string
1493       format: uuid
1494     semester:
1495       type: string
1496     year:
1497       type: integer
1498     section_number:
1499       type: string
1500   CourseSectionUpdate:
1501     type: object
1502     properties:
1503       subject_id:
1504         type: string
1505         format: uuid
1506       staff_id:
1507         type: string
1508         format: uuid
1509       semester:
1510         type: string
1511       year:
1512         type: integer
1513       section_number:
1514         type: string
1515   Schedule:
1516     type: object
1517     properties:
1518       id:
1519         type: string
1520         format: uuid
1521       section_id:
1522         type: string
1523         format: uuid
1524       room_id:
1525         type: string
1526         format: uuid
1527       day:
1528         type: string
1529         enum:
1530           - Monday
1531           - Tuesday
1532           - Wednesday
1533           - Thursday
1534           - Friday
1535           - Saturday
1536           - Sunday
1537       start_time:
1538         type: string
1539         format: time
1540       end_time:
1541         type: string
```

```
1542         format: time
1543     required:
1544     - id
1545     - section_id
1546     - room_id
1547     - day
1548     - start_time
1549     - end_time
1550 ScheduleCreate:
1551     type: object
1552     required:
1553     - section_id
1554     - room_id
1555     - day
1556     - start_time
1557     - end_time
1558     properties:
1559         section_id:
1560             type: string
1561             format: uuid
1562         room_id:
1563             type: string
1564             format: uuid
1565         day:
1566             type: string
1567             enum:
1568             - Monday
1569             - Tuesday
1570             - Wednesday
1571             - Thursday
1572             - Friday
1573             - Saturday
1574             - Sunday
1575         start_time:
1576             type: string
1577             format: time
1578         end_time:
1579             type: string
1580             format: time
1581 ScheduleUpdate:
1582     type: object
1583     properties:
1584         section_id:
1585             type: string
1586             format: uuid
1587         room_id:
1588             type: string
1589             format: uuid
1590         day:
1591             type: string
1592             enum:
1593             - Monday
1594             - Tuesday
1595             - Wednesday
1596             - Thursday
1597             - Friday
1598             - Saturday
1599             - Sunday
```

```
1600     start_time:
1601         type: string
1602         format: time
1603     end_time:
1604         type: string
1605         format: time
1606 Fee:
1607     type: object
1608     properties:
1609         id:
1610             type: string
1611             format: uuid
1612         student_id:
1613             type: string
1614             format: uuid
1615         amount_due:
1616             type: number
1617             format: float
1618         amount_paid:
1619             type: number
1620             format: float
1621         due_date:
1622             type: string
1623             format: date
1624         payment_status:
1625             type: string
1626             enum:
1627                 - paid
1628                 - unpaid
1629                 - overdue
1630     required:
1631         - id
1632         - student_id
1633         - amount_due
1634         - amount_paid
1635         - due_date
1636         - payment_status
1637 FeeCreate:
1638     type: object
1639     required:
1640         - student_id
1641         - amount_due
1642         - due_date
1643         - payment_status
1644     properties:
1645         student_id:
1646             type: string
1647             format: uuid
1648         amount_due:
1649             type: number
1650             format: float
1651         amount_paid:
1652             type: number
1653             format: float
1654         due_date:
1655             type: string
1656             format: date
1657         payment_status:
```

```
1658         type: string
1659         enum:
1660             - paid
1661             - unpaid
1662             - overdue
1663     FeeUpdate:
1664         type: object
1665         properties:
1666             amount_due:
1667                 type: number
1668                 format: float
1669             amount_paid:
1670                 type: number
1671                 format: float
1672             due_date:
1673                 type: string
1674                 format: date
1675             payment_status:
1676                 type: string
1677                 enum:
1678                     - paid
1679                     - unpaid
1680                     - overdue
1681     Enrollment:
1682         type: object
1683         properties:
1684             id:
1685                 type: string
1686                 format: uuid
1687             student_id:
1688                 type: string
1689                 format: uuid
1690             section_id:
1691                 type: string
1692                 format: uuid
1693             enrollment_status:
1694                 type: string
1695                 enum:
1696                     - Active
1697                     - Dropped
1698                     - Completed
1699         required:
1700             - id
1701             - student_id
1702             - section_id
1703             - enrollment_status
1704     EnrollmentCreate:
1705         type: object
1706         required:
1707             - student_id
1708             - section_id
1709             - enrollment_status
1710         properties:
1711             student_id:
1712                 type: string
1713                 format: uuid
1714             section_id:
1715                 type: string
```

```
1716         format: uuid
1717     enrollment_status:
1718         type: string
1719         enum:
1720             - Active
1721             - Dropped
1722             - Completed
1723 EnrollmentUpdate:
1724     type: object
1725     properties:
1726         student_id:
1727             type: string
1728             format: uuid
1729         section_id:
1730             type: string
1731             format: uuid
1732         enrollment_status:
1733             type: string
1734             enum:
1735                 - Active
1736                 - Dropped
1737                 - Completed
1738 SalaryProcess:
1739     type: object
1740     required:
1741         - staff_id
1742         - base_salary
1743         - payment_month
1744         - payment_year
1745     properties:
1746         staff_id:
1747             type: string
1748             format: uuid
1749         base_salary:
1750             type: number
1751             format: float
1752         allowances:
1753             type: number
1754             format: float
1755             default: 0
1756         deductions:
1757             type: number
1758             format: float
1759             default: 0
1760         payment_month:
1761             type: integer
1762             description: 1-12
1763         payment_year:
1764             type: integer
1765 SalaryResult:
1766     type: object
1767     properties:
1768         staff_id:
1769             type: string
1770             format: uuid
1771         total_salary:
1772             type: number
1773             format: float
```

```
1774     processed_at:
1775         type: string
1776         format: date-time
1777     base_salary:
1778         type: number
1779         format: float
1780     allowances:
1781         type: number
1782         format: float
1783     deductions:
1784         type: number
1785         format: float
1786     payment_month:
1787         type: integer
1788         description: 1-12
1789     payment_year:
1790         type: integer
1791 Room:
1792     type: object
1793     properties:
1794         id:
1795             type: string
1796             format: uuid
1797         room_number:
1798             type: string
1799         capacity:
1800             type: integer
1801         is_lab:
1802             type: boolean
1803     required:
1804         - id
1805         - room_number
1806         - capacity
1807         - is_lab
1808 RoomCreate:
1809     type: object
1810     required:
1811         - room_number
1812         - capacity
1813         - is_lab
1814     properties:
1815         room_number:
1816             type: string
1817         capacity:
1818             type: integer
1819         is_lab:
1820             type: boolean
1821 RoomUpdate:
1822     type: object
1823     properties:
1824         room_number:
1825             type: string
1826         capacity:
1827             type: integer
1828         is_lab:
1829             type: boolean
```

Listing B.1: OpenAPI Specification (openapi.yaml)